

Nền tảng nghiên cứu — Vision-Language Models for GUI Testing Automation

Step 1: Key Research (từ khóa cốt lõi của đề tài)

Từ tên đề tài “Ứng dụng Mô hình Thị giác - Ngôn ngữ (Vision-Language Model) trong Kiểm thử Tự động Giao diện Người dùng Phần mềm”, các cụm từ khóa nên dùng để tìm kiếm học thuật:

Nhóm	Từ khóa (tiếng Anh — dùng để search)
Công nghệ lõi	Vision-Language Model (VLM) , Multimodal LLM , GUI grounding , visual grounding
Bài toán	GUI testing automation , GUI agent , UI testing , mobile GUI testing , web GUI testing
Vấn đề cốt lõi	test maintenance , self-healing test , locator brittleness , flaky test
Đối tượng so sánh	locator-based testing , accessibility tree , DOM-based test automation
Mở rộng (bảo mật)	role-based access control testing , sensitive data masking , data privacy LLM API

Dùng các cụm này (kết hợp 2-3 từ một lần tìm) trên Google Scholar / IEEE Xplore / ACM Digital Library / Springer Link / arXiv sẽ ra đúng mạch tài liệu liên quan.

Step 2: 15 bài báo SOTA (2022-2026, venue uy tín — đã loại MDPI/tạp chí săn mồi)

Đã tìm và xác minh qua Google/arXiv/ACM DL/IEEE. Ưu tiên các venue Q1/hội nghị top ngành Software Engineering: **ICSE, FSE/ESEC-FSE (Proc. ACM Softw. Eng.), ISSTA, ICSME, IEEE TSE**. Vì đây là đồ án cá nhân, bạn không cần chia nhóm đọc — nhưng có thể đọc theo thứ tự ưu tiên (đánh dấu ★ = nên đọc kỹ nhất, phần còn lại đọc abstract + phần liên quan).

#	Bài báo	Venue / Năm	Vì sao liên quan
1 ★	Wang, J. et al. "Software Testing with Large Language Models: Survey, Landscape, and Vision"	IEEE TSE, 2024	Bài khảo sát tổng quan (Q1 journal) — dùng để viết Related Work, có bản đồ toàn cảnh lĩnh vực
2 ★	Liu, Z. et al. "Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions" (GPTDroid)	ICSE 2024	Agent LLM tương tác với app qua hội thoại — nền tảng của hướng "LLM as tester"
3 ★	Wang, S. et al. "Leveraging Large Vision-Language Model for Better Automatic Web GUI Testing"	ICSME 2024	Bài trực tiếp gần nhất với đề tài của bạn — VLM cho web GUI testing
4 ★	Liu, Z. et al. "Vision-driven Automated Mobile GUI Testing via Multimodal Large Language Model" (VisionDroid)	ISSTA/ASE 2024 (arXiv 2407.03037)	Dùng khả năng thị giác của VLM để phát hiện bug không gây crash — hướng dùng VLM để <i>verify</i> , không chỉ để <i>thao tác</i>
5	Liu, Z. et al. "Fill in the Blank: Context-aware Automated Text Input Generation for Mobile GUI Testing"	ICSE 2023	Tiền đề của GPTDroid, cho thấy quá trình LLM dần được đưa vào GUI testing từ 2023
6	Wang, C. et al. "LLMDroid: Enhancing Automated Mobile App GUI Testing Coverage with Large Language Model Guidance"	Proc. ACM Softw. Eng. (FSE), 2025	So sánh hiệu quả/chi phí giữa các phương pháp LLM-based (GPTDroid, DroidAgent, LLMDroid) — mẫu tốt cho cách thiết kế bảng so sánh chi phí trong Chương 4 của bạn
7	(nhóm tác giả) "AUITestAgent: Automatic Requirements Oriented GUI Function Testing"	arXiv 2407.09018 (workshop/FSE-related)	Sinh test tự động từ yêu cầu, có dùng CLIP-style vision encoder — tham khảo cách đánh giá độ chính xác định vị phần tử
8	(nhóm tác giả) "GUI Testing Arena: A Unified Benchmark for Advancing Autonomous"	arXiv 2412.18426 (benchmark)	Benchmark hợp nhất để đánh giá agent kiểm thử GUI — tham khảo cách thiết kế bộ chỉ số

	GUI Testing Agent"	paper)	(metric) thực nghiệm
9	<i>(nhóm tác giả)</i> "Temac: Multi-Agent Collaboration for Automated Web GUI Testing"	arXiv 2506.00520 (2025)	Multi-agent cho web GUI testing — có so sánh với các công cụ web testing khác (NaviQAte, feature-driven generation)
10	Shahbandeh, M. et al. "NaviQAte: Functionality-guided Web Application Navigation"	arXiv 2409.10741 (Mesbah group, UBC — nhóm uy tín trong web testing)	Điều hướng web app theo chức năng — liên quan đến phần "tự động lập kế hoạch hành động" của VLM agent
11	Alian, P. et al. "Feature-driven End-to-End Test Generation"	ICSE 2025	Nhóm nghiên cứu web-app-testing hàng đầu (Mesbah, UBC) — sinh test end-to-end, dùng làm baseline so sánh phương pháp truyền thống
12	<i>(nhóm tác giả)</i> "A Study of Using Multimodal LLMs for Non-Crash Functional Bug Detection in Android Apps"	arXiv 2407.19053 (ISSTA/ASE-track)	Đánh giá thực nghiệm khả năng multimodal LLM phát hiện lỗi chức năng — có bảng so sánh độ chính xác giữa các model, tham khảo tốt cho thiết kế Chương 4
13	<i>(nhóm tác giả)</i> "Intention-Based GUI Test Migration for Mobile Apps using Large Language Models"	Proc. ACM Softw. Eng. (FSE), 2025	Di trú (migrate) test case khi giao diện thay đổi — rất sát với khái niệm "chi phí bảo trì" bạn đang đo
14	<i>(nhóm tác giả)</i> "Guardian: A Runtime Framework for LLM-based UI Exploration"	ISSTA 2024	Framework runtime cho LLM khám phá UI — tham khảo kiến trúc pipeline
15	<i>(nhóm tác giả)</i> "Vision-Based Mobile App GUI Testing: A Survey"	2023 (survey, pre-LLM boom)	Khảo sát kiểm thử GUI dựa trên thị giác <i>trước</i> làn sóng LLM — giúp bạn thấy rõ vision-based testing đã có từ trước, LLM/VLM là bước phát triển tiếp theo

Cách đọc hiệu quả (vì đồ án cá nhân, không có nhóm chia nhau):

- Đọc kỹ Abstract + Introduction + Related Work của **cả 15 bài** (khoảng 2-3 trang/bài) → đủ để nắm bản đồ lĩnh vực.
- Đọc sâu phần Method + Evaluation của 4 bài đánh dấu ★ (#1, #2, #3, #4) — đây là 4 bài gần nhất với hướng đi của bạn.
- Ghi chú theo mẫu chuẩn cho từng bài: **Problem → Method → Result → Limitation** (1 dòng mỗi mục), để dễ tổng hợp thành Research Gap ở Step 3.

Lưu ý về truy cập: Các bài trên ACM DL/IEEE Xplore có thể cần VPN trường hoặc tài khoản thư viện số của UIT để tải full-text; các bản arXiv preprint tải được miễn phí trực tiếp.

Step 3: Research Gap — Research Question — Proposed Method

Research Gap

Từ 15 bài trên, có thể thấy 3 khoảng trống rõ ràng:

1. **Thiên lệch nền tảng:** Tuyệt đại đa số công trình LLM/VLM-for-GUI-testing (GPTDroid, VisionDroid, LLMDroid, AUITestAgent, Guardian) tập trung vào **Android mobile app**; nhóm còn lại (NaviQate, Feature-driven test generation, Temac) tập trung vào **web app**. Không có công trình nào trong 15 bài trên đánh giá một cách hệ thống việc **so sánh trực tiếp, có đo lường định lượng** giữa phương pháp locator-based truyền thống và phương pháp VLM-based, trên cùng một bộ test case, với biến thiên giao diện được kiểm soát chủ động (controlled UI variants) — phần lớn các bài đo hiệu quả qua *code coverage* hoặc *số bug phát hiện được*, không đo trực tiếp **độ bền vững khi giao diện thay đổi** (robustness under UI change) — vốn là lý do cốt lõi khiến VLM được đề xuất thay cho locator.
2. **Thiếu định lượng chi phí bảo trì:** Một số bài (LLMDroid, Intention-Based GUI Test Migration) có đề cập chi phí, nhưng đo theo hướng *chi phí vận hành agent* (token, thời gian chạy), chưa có công trình nào đo trực tiếp **chi phí bảo trì bộ test suite** (số dòng code/config phải sửa) khi so sánh cặp đôi giữa 2 cách tiếp cận trên cùng một ứng dụng.
3. **Bỏ ngỏ khía cạnh bảo mật:** Không có bài nào trong 15 bài này đề cập đến (a) rủi ro dữ liệu nhạy cảm bị lộ khi gửi ảnh chụp màn hình lên VLM API bên thứ ba, hay (b) khả năng tận dụng chính VLM để kiểm thử tính đúng đắn của giao diện theo phân quyền người dùng (role-based UI access). Đây là khoảng trống thực tế, không phải do khó về mặt kỹ thuật, mà do cộng đồng nghiên cứu SE-testing hiện chưa chú ý tới góc bảo mật này — phù hợp làm đóng góp bổ sung cho một đề án cử nhân.

Research Question

- **RQ1:** So với kiểm thử dựa trên locator truyền thống, kiểm thử dựa trên VLM cải thiện độ bền vững (tỉ lệ test pass) đến mức nào khi giao diện thay đổi (theme, bố cục, biểu tượng)?
- **RQ2:** Chi phí bảo trì bộ test suite (số dòng code/config cần sửa, thời gian, chi phí gọi API) của 2 phương pháp khác nhau như thế nào khi giao diện thay đổi?
- **RQ3:** VLM có thể được tận dụng để phát hiện lỗi hiển thị giao diện theo phân quyền người dùng (role-based UI access) với độ chính xác ra sao, so với kiểm tra thủ công/locator-based?
- **RQ4:** Việc che giấu (masking) dữ liệu nhạy cảm trên ảnh chụp màn hình trước khi gửi lên VLM API ảnh hưởng thế nào đến độ chính xác định vị phần tử của VLM (đánh đổi bảo mật – độ chính xác)?

Proposed Method

Nghiên cứu thực nghiệm so sánh (empirical comparative study), gồm 4 khối chính, tương ứng đúng với 4 Nội dung đã có trong đề cương:

1. Xây dựng ứng dụng thực nghiệm + các biến thể giao diện có kiểm soát (controlled UI variants).
2. Xây dựng 2 bộ test song song: locator-based (Playwright/Selenium) và VLM-based (Midscene.js), cùng chức năng, cùng số lượng test case.
3. Chạy cả 2 bộ trên giao diện gốc và các biến thể, thu thập số liệu định lượng theo RQ1, RQ2.
4. Xây dựng 2 khảo sát mở rộng (masking + role-based UI testing) trả lời RQ3, RQ4.

Step 4: 5 Contribution chốt để làm

1. **Bộ dữ liệu thực nghiệm tái sử dụng được:** một ứng dụng mẫu + tập biến thể giao diện có kiểm soát (2-3 biến thể theme/bố cục/icon), công khai trên GitHub kèm báo cáo — dùng làm benchmark cho các nghiên cứu GUI testing robustness sau này.
2. **Pipeline/framework đánh giá thực nghiệm so sánh** giữa kiểm thử VLM-based và locator-based, đo đồng thời 3 trục: độ bền vững (robustness), chi phí bảo trì (maintenance cost), và chi phí vận hành (thời gian/token) — đóng góp về mặt **phương pháp đánh giá** (không phải công cụ mới).

3. **Bộ số liệu thực nghiệm định lượng** trả lời trực tiếp RQ1, RQ2 — là bảng chứng thực nghiệm đầu tiên (trong phạm vi khảo sát 15 bài trên) đo trực tiếp đánh đổi robustness vs. cost giữa 2 phương pháp trên cùng một ứng dụng, cùng một bộ test case.
4. **Cơ chế tiền xử lý ảnh che giấu dữ liệu nhạy cảm** trước khi gửi lên VLM API, cùng đánh giá định lượng về đánh đổi giữa mức độ bảo mật và độ chính xác định vị phân tử (trả lời RQ4) — khoảng trống chưa được nghiên cứu học thuật nào trong 15 bài trên đề cập.
5. **Bộ kịch bản kiểm thử phân quyền hiển thị giao diện (role-based UI access) bằng VLM**, cùng đánh giá tỉ lệ phát hiện lỗi so với phương pháp truyền thống (trả lời RQ3) — mở rộng ứng dụng của VLM từ “kiểm thử chức năng” sang “kiểm thử liên quan bảo mật”, một hướng đi mới so với các công trình hiện có.

File này nên được cập nhật liên tục trong quá trình đọc tài liệu — mỗi khi đọc xong 1 bài, ghi thêm 1 dòng ghi chú Problem/Method/Result/Limitation vào bảng Step 2 để tích lũy dần cho phần Related Work của báo cáo.